## Ex. No: 1

```
CREATE TABLE Student (StudentID int PRIMARY KEY, Name varchar(30), Age
int);
```

**Output:**

*Table created.*

```
INSERT INTO Student VALUES (1, 'Alice', 20);
INSERT INTO Student VALUES (2, 'Bob', 22);
INSERT INTO Student VALUES (3, 'Charlie', 21);
INSERT INTO Student VALUES (4, 'David', 19);
```

**Output:**

*4 rows created.*

```
CREATE TABLE Courses (CourseID int PRIMARY KEY, CourseName varchar(20));
```

**Output:**

*Table created.*

```
INSERT INTO Courses VALUES (101, 'Database Management');
INSERT INTO Courses VALUES (102, 'Algorithms');
INSERT INTO Courses VALUES (103, 'Web Development');
```

**Output:**

*3 rows created.*

```
CREATE TABLE Enrollments (StudentID int REFERENCES Student(StudentID),
CourseID int REFERENCES Courses(CourseID));
```

**Output:**

*Table created.*

```
INSERT INTO Enrollments VALUES (1, 101);
INSERT INTO Enrollments VALUES (1, 102);
INSERT INTO Enrollments VALUES (2, 102);
INSERT INTO Enrollments VALUES (3, 101);
INSERT INTO Enrollments VALUES (3, 103);
INSERT INTO Enrollments VALUES (4, 103);
```

**Output:**

*6 rows created.*

SELECT * FROM Student;

**Output:**

| StudentID | Name | Age |
|---|---|---|
| 1 | Alice | 20 |
| 2 | Bob | 22 |
| 3 | Charlie | 21 |
| 4 | David | 19 |

SELECT Name, Age FROM Student WHERE Age > 20;

**Output:**

| Name | Age |
|---|---|
| Bob | 22 |
| Charlie | 21 |

SELECT Name FROM Student WHERE StudentID IN (SELECT StudentID FROM Enrollments WHERE CourseID = (SELECT CourseID FROM Courses WHERE CourseName = 'Database Management'));

**Output:**

| Name |
|---|
| Alice |
| Charlie |

SELECT CourseID, CourseName FROM Courses WHERE CourseID IN (SELECT CourseID FROM Enrollments GROUP BY CourseID HAVING COUNT(*) > 1);

**Output:**

| CourseID | CourseName |
|---|---|
| 101 | Database Management |
| 102 | Algorithms |
| 103 | Web Development |

SELECT AVG(Age) AS AverageAge FROM Student;

**Output:**

| AverageAge |
|---|
| 20.5 |

```
SELECT Name, Age FROM Student WHERE Age > (SELECT AVG(Age) FROM
Student);
```

**Output:**

| Name | Age |
|---|---|
| Bob | 22 |
| Charlie | 21 |

## Ex. No: 2

CREATE TABLE Student (StudentID int, Name varchar(30), Age int);

**Output:**

*Table created.*

INSERT INTO Student VALUES (1, 'Alice', 20);
INSERT INTO Student VALUES (2, 'Bob', 22);
INSERT INTO Student VALUES (3, 'Charlie', 21);

**Output:**

*3 rows created.*

CREATE TABLE Courses (CourseID int, CourseName varchar(20));

**Output:**

*Table created.*

INSERT INTO Courses VALUES (101, 'Math');
INSERT INTO Courses VALUES (102, 'English');
INSERT INTO Courses VALUES (103, 'History');

**Output:**

*3 rows created.*

CREATE TABLE Enrollments (EnrollmentID int, StudentID int, CourseID int, Grade varchar(5));

**Output:**

*Table created.*

INSERT INTO Enrollments VALUES (1, 1, 101, 'A');
INSERT INTO Enrollments VALUES (2, 1, 102, 'B');
INSERT INTO Enrollments VALUES (3, 2, 101, 'A-');
INSERT INTO Enrollments VALUES (4, 3, 103, 'B+');
INSERT INTO Enrollments VALUES (5, 3, 102, 'A');

**Output:**

*5 rows created.*

**INNER JOIN:**

SELECT Student.StudentID, Student.Name, Student.Age, Courses.CourseID, Courses.CourseName, Enrollments.Grade FROM Student INNER JOIN Enrollments ON Student.StudentID = Enrollments.StudentID INNER JOIN Courses ON Enrollments.CourseID = Courses.CourseID;

**Output:**

| StudentID | Name | Age | CourseID | CourseName | Grade |
|---|---|---|---|---|---|
| 1 | Alice | 20 | 101 | Math | A |
| 1 | Alice | 20 | 102 | English | B |
| 2 | Bob | 22 | 101 | Math | A- |
| 3 | Charlie | 21 | 103 | History | B+ |
| 3 | Charlie | 21 | 102 | English | A |

**LEFT JOIN:**

SELECT Student.StudentID, Student.Name, Student.Age, Courses.CourseID, Courses.CourseName, Enrollments.Grade FROM Student LEFT JOIN Enrollments ON Student.StudentID = Enrollments.StudentID LEFT JOIN Courses ON Enrollments.CourseID = Courses.CourseID;

**Output:**

| StudentID | Name | Age | CourseID | CourseName | Grade |
|---|---|---|---|---|---|
| 1 | Alice | 20 | 101 | Math | A |
| 1 | Alice | 20 | 102 | English | B |
| 2 | Bob | 22 | 101 | Math | A- |
| 3 | Charlie | 21 | 103 | History | B+ |
| 3 | Charlie | 21 | 102 | English | A |

**RIGHT JOIN:**

SELECT Student.StudentID, Student.Name, Student.Age, Courses.CourseID, Courses.CourseName, Enrollments.Grade FROM Courses RIGHT JOIN Enrollments ON Courses.CourseID = Enrollments.CourseID RIGHT JOIN Student ON Enrollments.StudentID = Student.StudentID;

**Output:**

| StudentID | Name | Age | CourseID | CourseName | Grade |
|---|---|---|---|---|---|
| 1 | Alice | 20 | 101 | Math | A |
| 1 | Alice | 20 | 102 | English | B |
| 3 | Charlie | 21 | 103 | History | B+ |
| 2 | Bob | 22 | 101 | Math | A- |
| 3 | Charlie | 21 | 102 | English | A |

## Ex. No: 3

CREATE SEQUENCE student_seq START WITH 1 INCREMENT BY 1;

**Output:**

*Sequence created.*

CREATE TABLE students (student_id INT PRIMARY KEY, student_name VARCHAR(100), student_email VARCHAR(100));

**Output:**

*Table created.*

INSERT INTO students (student_id, student_name, student_email) VALUES (student_seq.NEXTVAL, 'Alice Johnson', 'alice@example.com');
INSERT INTO students (student_id, student_name, student_email) VALUES (student_seq.NEXTVAL, 'Bob Smith', 'bob@example.com');
INSERT INTO students (student_id, student_name, student_email) VALUES (student_seq.NEXTVAL, 'Charlie Brown', 'charlie@example.com');

**Output:**

*3 rows created.*

CREATE VIEW student_view AS SELECT student_id, student_name, student_email FROM students;

**Output:**

*View created.*

INSERT INTO students (student_id, student_name, student_email) VALUES (student_seq.NEXTVAL, 'Diana Prince', 'diana@example.com');

**Output:**

*1 row created.*

UPDATE students SET student_email = 'new_bob@example.com' WHERE student_name = 'Bob Smith';

**Output:**

*1 row updated.*

DELETE FROM students WHERE student_name = 'Charlie Brown';

**Output:**

*1 row deleted.*

CREATE INDEX idx_student_email ON students (student_email);

**Output:**

*Index created.*

SELECT * FROM student_view;

**Output:**

| student_id | student_name | student_email |
|---|---|---|
| 1 | Alice Johnson | alice@example.com |
| 2 | Bob Smith | new_bob@example.com |
| 4 | Diana Prince | diana@example.com |

## Ex. No: 4

```
CREATE TABLE customers (id NUMBER PRIMARY KEY, name VARCHAR2(30),
age NUMBER, address VARCHAR2(50), salary NUMBER(10, 2));
```

**Output:**

*Table created.*

```
INSERT INTO customers VALUES (1, 'Ramesh', 32, 'Ahmadabad', 2000.00);
INSERT INTO customers VALUES (2, 'Khilan', 25, 'Delhi', 1500.00);
INSERT INTO customers VALUES (3, 'Kaushik', 23, 'Kota', 2000.00);
INSERT INTO customers VALUES (4, 'Chaitali', 25, 'Mumbai', 6500.00);
INSERT INTO customers VALUES (5, 'Hardik', 27, 'Bhopal', 8500.00);
INSERT INTO customers VALUES (6, 'Komal', 22, 'MP', 4500.00);
```

**Output:**

*6 rows created.*

**Implicit Cursor:**

```
SET SERVEROUTPUT ON;
DECLARE
 total_rows number(2);
BEGIN
   UPDATE customers SET salary = salary + 500;
   IF sql%notfound THEN
      dbms_output.put_line('no customers selected');
   ELSIF sql%found THEN
    total_rows := sql%rowcount;
      dbms_output.put_line(total_rows || ' customers selected ');
   END IF;
END;
/
```

**Output:**

*6 customers selected*

*PL/SQL procedure successfully completed.*

```
SELECT * FROM customers;
```

**Output:**

| ID | NAME | AGE | ADDRESS | SALARY |
|---|---|---|---|---|
| 1 | Ramesh | 32 | Ahmadabad | 2500.00 |
| 2 | Khilan | 25 | Delhi | 2000.00 |
| 3 | Kaushik | 23 | Kota | 2500.00 |
| 4 | Chaitali | 25 | Mumbai | 7000.00 |
| 5 | Hardik | 27 | Bhopal | 9000.00 |
| 6 | Komal | 22 | MP | 5000.00 |

**Explicit Cursor:**

```
DECLARE
   c_id customers.id%type;
 c_name customers.name%type;
 c_addr customers.address%type;
   CURSOR c_customers IS SELECT id, name, address FROM customers;
BEGIN
   OPEN c_customers;
   LOOP
      FETCH c_customers INTO c_id, c_name, c_addr;
      EXIT WHEN c_customers%notfound;
      dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);
   END LOOP;
   CLOSE c_customers;
END;
/
```

**Output:**

*PL/SQL procedure successfully completed.*

| ID | NAME | ADDRESS |
|---|---|---|
| 1 | Ramesh | Ahmadabad |
| 2 | Khilan | Delhi |
| 3 | Kaushik | Kota |
| 4 | Chaitali | Mumbai |
| 5 | Hardik | Bhopal |
| 6 | Komal | MP |

## Ex. No: 5

**Procedure:**

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE PROCEDURE Sum_Proc(a IN number, b IN number) IS
 c number;
BEGIN
 c := a + b;
   dbms_output.put_line('Sum of two nos = ' || c);
END Sum_Proc;
/
```

**Output:**

*Procedure created.*

```
DECLARE
 x number;
 y number;
BEGIN
 x := &x;
 y := &y;
 Sum_Proc(x, y);
END;
/
```

**Output:**

*Enter value for x: 10*
*Enter value for y: 20*
*Sum of two nos = 30*
*PL/SQL procedure successfully completed.*

**Function:**

```
SET SERVEROUTPUT ON;
CREATE OR REPLACE FUNCTION Sum_Func(a IN number, b IN number)
RETURN number IS
 c number;
BEGIN
 c := a + b;
   RETURN c;
END Sum_Func;
```

**Output:**

*Function created.*

```
DECLARE
 no1 number;
 no2 number;
 result number;
BEGIN
 no1 := &no1;
 no2 := &no2;
 result := Sum_Func(no1, no2);
   dbms_output.put_line('Sum of two nos = ' || result);
END;
/
```

**Output:**

*Enter value for no1: 5*
*Enter value for no2: 5*
*Sum of two nos = 10*
*PL/SQL procedure successfully completed.*

## Ex. No: 6

```
CREATE TABLE customer (sid NUMBER PRIMARY KEY, sname
VARCHAR2(30), salary NUMBER);
```

**Output:**

*Table created.*

```
INSERT INTO customer VALUES (1, 'Aarthi', 300);
INSERT INTO customer VALUES (2, 'Ramya', 500);
INSERT INTO customer VALUES (3, 'Sakthi', 670);
```

**Output:**

*3 rows created.*

**BEFORE UPDATE Trigger:**

```
CREATE OR REPLACE TRIGGER up_classd
BEFORE UPDATE ON customer
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('new value is ' || :new.salary);
   DBMS_OUTPUT.PUT_LINE('old value is ' || :old.salary);
END;
/
```

**Output:**

*Trigger created.*

```
UPDATE customer SET salary = 500 WHERE sid = 1;
```

**Output:**

*new value is 500*
*old value is 300*
*1 row updated.*

```
SELECT * FROM customer;
```

**Output:**

| SID | SNAME | SALARY |
|---|---|---|
| 1 | Aarthi | 500 |
| 2 | Ramya | 500 |
| 3 | Sakthi | 670 |

**BEFORE DELETE Trigger:**

```
CREATE OR REPLACE TRIGGER del_classb
BEFORE DELETE ON customer
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('row deleted');
END;
/
```

**Output:**

*Trigger created.*

```
DELETE FROM customer WHERE sid = 1;
```

**Output:**

*row deleted*
*1 row deleted.*

```
SELECT * FROM customer;
```

**Output:**

| SID | SNAME | SALARY |
|---|---|---|
| 2 | Ramya | 500 |
| 3 | Sakthi | 670 |

```
CREATE TABLE classb (sid NUMBER, sname VARCHAR2(20), dept
VARCHAR2(10), stotal NUMBER, grade VARCHAR2(2));
```

**Output:**

*Table created.*

**BEFORE INSERT Trigger:**

```
CREATE OR REPLACE TRIGGER ins_classb
BEFORE INSERT ON classb
FOR EACH ROW
DECLARE
 InvTot EXCEPTION;
BEGIN
   IF :new.stotal > 1000 THEN
      RAISE InvTot;
   END IF;
```

```
EXCEPTION
   WHEN InvTot THEN
      RAISE_APPLICATION_ERROR(-20000, 'Total not valid');
END;
/
```

**Output:**

*Trigger created.*

```
INSERT INTO classb VALUES (5, 'vino', 'it', 500, 'a');
```

**Output:**

*1 row created.*

```
INSERT INTO classb VALUES (6, 'jana', 'it', 2000, 'a');
```

**Output:**

*ERROR at line 1:*
*ORA-20000: Total not valid*
*ORA-06512: at "SCOTT.INS_CLASSB", line 11*
*ORA-04088: error during execution of trigger 'SCOTT.INS_CLASSB'*

## Ex. No: 7

CREATE TABLE Authors (AuthorID INT PRIMARY KEY, FirstName VARCHAR2(50), LastName VARCHAR2(50));

**Output:**

*Table created.*

INSERT INTO Authors (AuthorID, FirstName, LastName) VALUES (1, 'George', 'Orwell');
INSERT INTO Authors (AuthorID, FirstName, LastName) VALUES (2, 'Aldous', 'Huxley');
INSERT INTO Authors (AuthorID, FirstName, LastName) VALUES (3, 'J.K.', 'Rowling');

**Output:**

*3 rows created.*

SELECT * FROM Authors;

**Output:**

| AuthorID | FirstName | LastName |
|---|---|---|
| 1 | George | Orwell |
| 2 | Aldous | Huxley |
| 3 | J.K. | Rowling |

CREATE TABLE Books (BookID INT PRIMARY KEY, Title VARCHAR2(100), Genre VARCHAR2(50), PublicationYear INT);

**Output:**

*Table created.*

INSERT INTO Books (BookID, Title, Genre, PublicationYear) VALUES (1, '1984', 'Dystopian', 1949);
INSERT INTO Books (BookID, Title, Genre, PublicationYear) VALUES (2, 'Brave New World', 'Dystopian', 1932);
INSERT INTO Books (BookID, Title, Genre, PublicationYear) VALUES (3, 'Harry Potter and the Sorcerers Stone', 'Fantasy', 1997);

**Output:**

*3 rows created.*

SELECT * FROM Books;

**Output:**

| BookID | Title | Genre | PublicationYear |
|---|---|---|---|
| 1 | 1984 | Dystopian | 1949 |
| 2 | Brave New World | Dystopian | 1932 |
| 3 | Harry Potter and the Sorcerers Stone | Fantasy | 1997 |

```
CREATE TABLE BookAuthors (BookID INT,
  AuthorID INT,PRIMARY KEY (BookID, AuthorID),
  FOREIGN KEY (BookID) REFERENCES Books(BookID),
  FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID)
);
```

**Output:**

*Table created.*

```
INSERT INTO BookAuthors (BookID, AuthorID) VALUES (1, 1);
INSERT INTO BookAuthors (BookID, AuthorID) VALUES (2, 2);
INSERT INTO BookAuthors (BookID, AuthorID) VALUES (3, 3);
```

**Output:**

*3 rows created.*

```
SELECT * FROM BookAuthors;
```

**Output:**

| BookID | AuthorID |
|---|---|
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |

```
CREATE TABLE Borrowers (
  BorrowerID INT PRIMARY KEY,
  FirstName VARCHAR2(50),
  LastName VARCHAR2(50),
  MembershipDate DATE
);
```

**Output:**

*Table created.*

```
INSERT INTO Borrowers (BorrowerID, FirstName, LastName, MembershipDate)
VALUES (1, 'John', 'Doe', TO_DATE('2023-01-01', 'YYYY-MM-DD'));
INSERT INTO Borrowers (BorrowerID, FirstName, LastName, MembershipDate)
VALUES (2, 'Jane', 'Smith', TO_DATE('2023-02-15', 'YYYY-MM-DD'));
```

**Output:**

*2 rows created.*

```
SELECT * FROM Borrowers;
```

**Output:**

| BorrowerID | FirstName | LastName | MembershipDate |
|---|---|---|---|
| 1 | John | Doe | 2023-01-01 |
| 2 | Jane | Smith | 2023-02-15 |

```
CREATE TABLE BorrowedBooks (BorrowerID INT, BookID INT,
 BorrowedDate DATE, ReturnDate DATE, PRIMARY KEY (BorrowerID, BookID),
 FOREIGN KEY (BorrowerID) REFERENCES Borrowers(BorrowerID),
 FOREIGN KEY (BookID) REFERENCES Books(BookID));
```

**Output:**

*Table created.*

```
INSERT INTO BorrowedBooks (BorrowerID, BookID, BorrowedDate, ReturnDate)
VALUES (1, 1, TO_DATE('2023-03-01', 'YYYY-MM-DD'), TO_DATE('2023-03-15',
'YYYY-MM-DD'));
INSERT INTO BorrowedBooks (BorrowerID, BookID, BorrowedDate, ReturnDate)
VALUES (2, 3, TO_DATE('2023-03-05', 'YYYY-MM-DD'), TO_DATE('2023-03-20',
'YYYY-MM-DD'));
```

**Output:**

*2 rows created.*

```
SELECT * FROM BorrowedBooks;
```

**Output:**

| BorrowerID | BookID | BorrowedDate | ReturnDate |
|---|---|---|---|
| 1 | 1 | 2023-03-01 | 2023-03-15 |
| 2 | 3 | 2023-03-05 | 2023-03-20 |

## Ex. No: 8

**System-defined exception:**

```
SET SERVEROUTPUT ON;
DECLARE
   c_id customers.id%type := 5;
 c_name customers.name%type;
 c_addr customers.address%type;
BEGIN
   SELECT name, address INTO c_name, c_addr FROM customers WHERE id = c_id;
 dbms_output.put_line('name: ' || c_name);
 dbms_output.put_line('address: ' || c_addr);
EXCEPTION
   WHEN no_data_found THEN
    dbms_output.put_line('no such customer!');
   WHEN others THEN
    dbms_output.put_line('error!');
END;
/
```

**Output:**

*name: Hardik*
*address: Bhopal*
*PL/SQL procedure successfully completed.*

**User-defined exception:**

```
DECLARE
   c_id customers.id%type := &cc_id;
 c_name customers.name%type;
 c_addr customers.address%type;
   ex_invalid_id EXCEPTION;
BEGIN
   IF c_id <= 0 THEN
      RAISE ex_invalid_id;
   ELSE
      SELECT name, address INTO c_name, c_addr FROM customers WHERE id =
c_id;
    DBMS_OUTPUT.PUT_LINE('Name: ' || c_name);
    DBMS_OUTPUT.PUT_LINE('Address: ' || c_addr);
```

```
    END IF;
  EXCEPTION
    WHEN ex_invalid_id THEN
     dbms_output.put_line('ID must be greater than zero!');
    WHEN no_data_found THEN
     dbms_output.put_line('No such customer!');
    WHEN others THEN
     dbms_output.put_line('Error!');
  END;
  /
```

**Output:**

*Enter value for cc_id: -6*
*ID must be greater than zero!*

*PL/SQL procedure successfully completed.*

## Ex. No: 9

**Key Value Pair Based**

- Data is stored in key/value pairs. It is designed in such a way to handle lots of data and heavy load.

- Key-value pair storage databases store data as a hash table where each key is unique, and the value can be a JSON, BLOB (Binary Large Objects), string, etc.

- For example, a key-value pair may contain a key like "Website" associated with a value like "Guru99".

| Key | Value |
|---|---|
| Name | Joe Bloggs |
| Age | 42 |
| Occupation | Stunt Double |
| Height | 175cm |
| Weight | 77kg |

- It is one of the most basic NoSQL database example. This kind of NoSQL database is used as a collection, dictionaries, associative arrays, etc. Key value stores help the developer to store schema-less data. They work best for shopping cart contents.

- Redis, Dynamo, Riak are some NoSQL examples of key-value store DataBases. They are all based on Amazon's Dynamo paper.

**Column-based**

- Column-oriented databases work on columns and are based on BigTable paper by Google. Every column is treated separately. Values of single column databases are stored contiguously.

| ColumnFamily | | | |
|---|---|---|---|
| Row Key | Column Name | | |
| | Key | Key | Key |
| | Value | Value | Value |
| | Column Name | | |
| | Key | Key | Key |
| | Value | Value | Value |

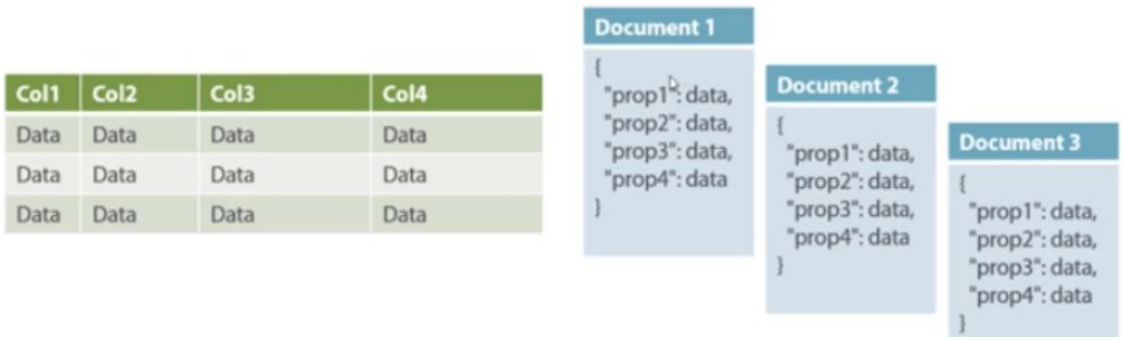
- They deliver high performance on aggregation queries like SUM, COUNT, AVG, MIN etc. as the data is readily available in a column.

- Column-based NoSQL databases are widely used to manage data warehouses, business intelligence, CRM, Library card catalogs.

- HBase, Cassandra, HBase, Hypertable are NoSQL query examples of column based database.

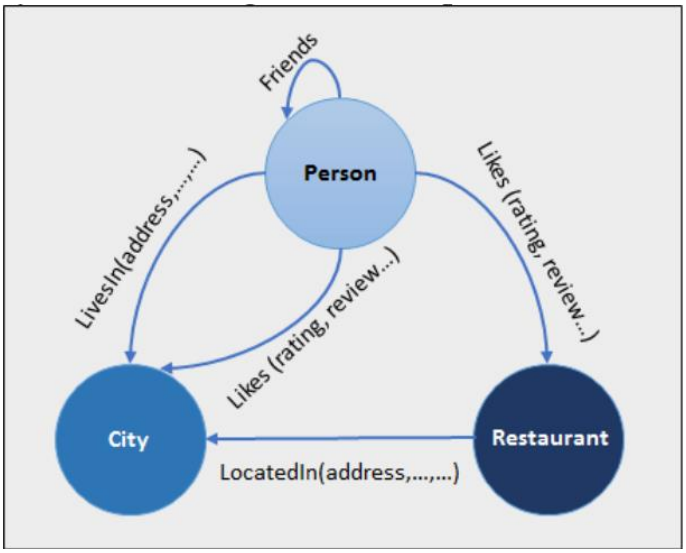
**Document-Oriented**

- Document-Oriented NoSQL DB stores and retrieves data as a key value pair but the value part is stored as a document. The document is stored in JSON or XML formats. The value is understood by the DB and can be queried.




**Graph-Based**

- A graph type database stores entities as well the relations amongst those entities. The entity is stored as a node with the relationship as edges. An edge gives a relationship between nodes. Every node and edge has a unique identifier.



## Ex. No: 10

```
use library;
  db.authors.insertMany([
     { "AuthorID": 1, "FirstName": "George", "LastName": "Orwell" },
     { "AuthorID": 2, "FirstName": "Aldous", "LastName": "Huxley" },
     { "AuthorID": 3, "FirstName": "J.K.", "LastName": "Rowling" }
]);
```

**Output:**

*{ "acknowledged": true, "insertedIds": [1, 2, 3] }*

```
db.books.insertMany([
   { "BookID": 1, "Title": "1984", "Genre": "Dystopian", "PublicationYear": 1949, "Authors": [1] },
   { "BookID": 2, "Title": "Brave New World", "Genre": "Dystopian", "PublicationYear": 1932, "Authors": [2] },
   { "BookID": 3, "Title": "Harry Potter and the Sorcerer's Stone", "Genre": "Fantasy", "PublicationYear": 1997, "Authors": [3] }
]);
```

**Output:**

```
{ "acknowledged": true, "insertedIds": [1, 2, 3] }
db.borrowers.insertMany([
   { "BorrowerID": 1, "FirstName": "John", "LastName": "Doe", "MembershipDate": new Date("2023-01-01") },
   { "BorrowerID": 2, "FirstName": "Jane", "LastName": "Smith", "MembershipDate": new Date("2023-02-15") }
]);
```

**Output:**

```
{ "acknowledged": true, "insertedIds": [1, 2] }
db.borrowedBooks.insertMany([
   { "BorrowerID": 1, "BookID": 1, "BorrowedDate": new Date("2023-03-01"), "ReturnDate": new Date("2023-03-15") },
   { "BorrowerID": 2, "BookID": 3, "BorrowedDate": new Date("2023-03-05"), "ReturnDate": new Date("2023-03-20") }
]);
```

**Output:**

{ "acknowledged": true, "insertedIds": [1, 2] }

db.authors.updateOne({ "AuthorID": 1 }, { $set: { "LastName": "Smith" } });

**Output:**

*{ "acknowledged": true, "matchedCount": 1, "modifiedCount": 1 }*

db.authors.find().pretty();

**Output:**

*{ "_id": ObjectId("..."), "AuthorID": 1, "FirstName": "George", "LastName": "Smith" }*
*{ "_id": ObjectId("..."), "AuthorID": 2, "FirstName": "Aldous", "LastName": "Huxley" }*
*{ "_id": ObjectId("..."), "AuthorID": 3, "FirstName": "J.K.", "LastName": "Rowling" }*

db.authors.deleteOne({ "AuthorID": 1 });

**Output:**

*{ "acknowledged": true, "deletedCount": 1 }*

db.authors.find().pretty();

**Output:**

*{ "_id": ObjectId("..."), "AuthorID": 2, "FirstName": "Aldous", "LastName": "Huxley" }*
*{ "_id": ObjectId("..."), "AuthorID": 3, "FirstName": "J.K.", "LastName": "Rowling" }*

## Ex. No: 11

CREATE KEYSPACE library WITH REPLICATION = { 'class': 'SimpleStrategy', 'replication_factor': 3 };

**Output:**

*Keyspace created.*

USE library;

**Output:**

*Context switched to library.*

```
CREATE TABLE authors (
 AuthorID int PRIMARY KEY,
 FirstName text,
 LastName text
);
```

**Output:**

*Table created.*

INSERT INTO authors (AuthorID, FirstName, LastName) VALUES (1, 'George', 'Orwell');
INSERT INTO authors (AuthorID, FirstName, LastName) VALUES (2, 'Aldous', 'Huxley');
INSERT INTO authors (AuthorID, FirstName, LastName) VALUES (3, 'J.K.', 'Rowling');

**Output:**

*3 rows inserted.*

SELECT * FROM authors;

**Output:**

| authorid | firstname | lastname |
|---|---|---|
| 1 | George | Orwell |
| 2 | Aldous | Huxley |
| 3 | J.K. | Rowling |

UPDATE authors SET LastName = 'Smith' WHERE AuthorID = 1;
SELECT * FROM authors WHERE AuthorID = 1;

**Output:**

| authorid | firstname | lastname |
|---|---|---|
| 1 | George | Smith |

```
DELETE FROM authors WHERE AuthorID = 1;

SELECT * FROM authors WHERE AuthorID = 1;
```

**Output:**

*(Empty result set)*

## Ex. No: 12

**Case Study:**

Banking is one of the most familiar real-life database applications. A bank must keep a record of its customers and their accounts, make sure every account balance stays consistent, and log each deposit and withdrawal the moment it happens. This case study models a simplified retail banking system, in which a customer holds one or more accounts and every movement of money is recorded as a transaction, and shows how a desktop front end can carry out these operations directly against the database.

**Entities and Relationships:**

The design is built from three entities. Customer (custid, cname, phone) represents the account holder. Account (accno, custid, balance) belongs to exactly one customer through a foreign key on custid, and its balance is constrained never to fall below zero. Transactions (txnid, accno, txntype, amount, balance_after, txntime) records every operation performed on an account, whether OPEN, DEPOSIT or WITHDRAW, and is linked to Account by a foreign key. The relationships are therefore one Customer to many Accounts, and one Account to many Transactions.

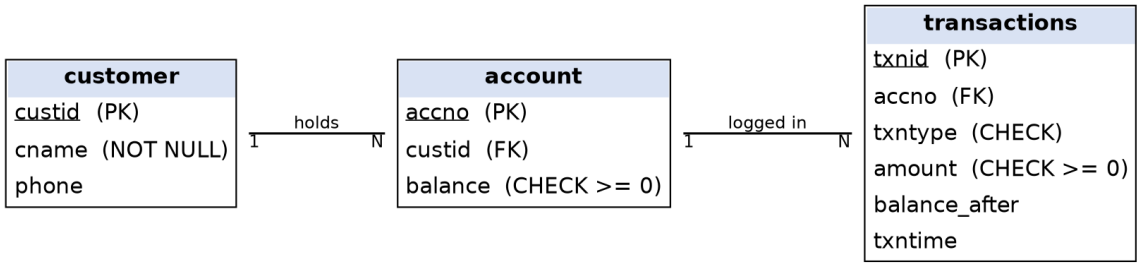



*Fig. 12.1: ER diagram (customer 1:N account 1:N transactions).*

The tables are in Third Normal Form: customer details are stored once, and each transaction refers to its account only through accno. transactions.balance_after is a deliberate snapshot of the balance, so a statement can be shown without replaying the ledger.

**Features:**

The application supports six operations: Insert (open a new account together with its customer), Update (edit a customer's name and phone), Delete (close an account and remove its transaction history), Deposit and Withdraw (each guarded by an insufficient-balance check), and History (view the transaction log of a single account or of all accounts). A balance can change only through Deposit or Withdraw, so the account balance and the transaction ledger always stay in step.

**Database Connectivity:**

The front end is built with Tkinter, Python's standard GUI toolkit, and connects to the SQLite database through Python's built-in sqlite3 module. Every button on the window issues SQL over this connection: for example, Deposit runs an UPDATE on account and then an INSERT into transactions within a single database transaction, so both writes succeed together or neither is applied. Referential integrity is switched on with PRAGMA foreign_keys = ON, and table-level rules such as CHECK (balance >= 0) are enforced by the database engine itself, not by the interface. The schema and the complete program follow.

Architecture: BankApp (Tkinter) is the presentation layer and only calls BankDB methods; BankDB is the application layer that validates input and controls transactions; the sqlite3 driver (the role JDBC or ODBC plays in other stacks) carries SQL to the SQLite DBMS. Here all layers run on one computer; a real bank would put an application server between the app and the database, reached over HTTPS.

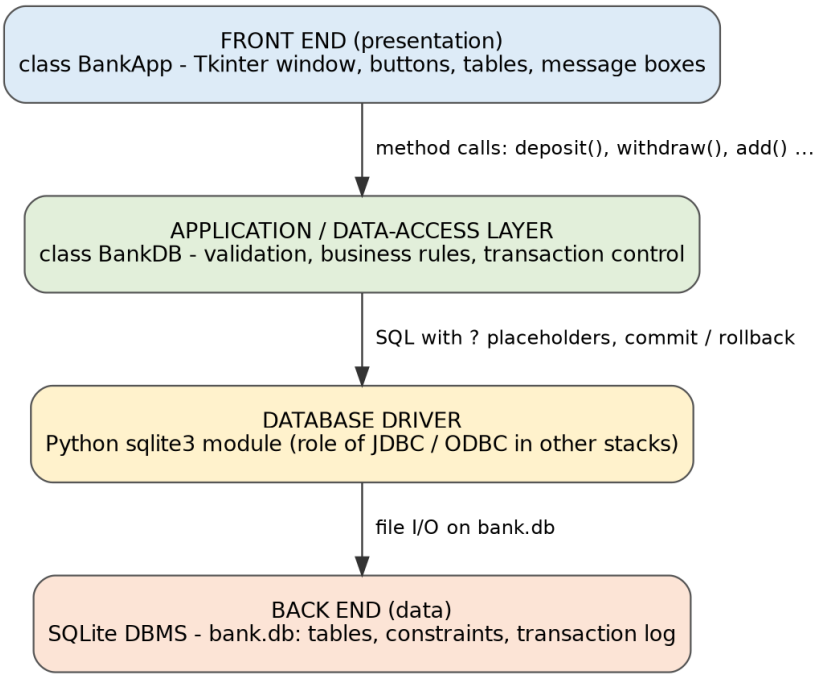



*Fig. 12.2: Layers and connectivity path from the GUI to the database.*

**Schema (schema.sql):**

```
CREATE TABLE IF NOT EXISTS customer (
    custid INTEGER PRIMARY KEY AUTOINCREMENT,
    cname  TEXT NOT NULL,
    phone  TEXT
);
```

```
CREATE TABLE IF NOT EXISTS account (
   accno   INTEGER PRIMARY KEY,
   custid  INTEGER NOT NULL REFERENCES customer(custid),
   balance REAL    NOT NULL DEFAULT 0 CHECK (balance >= 0)
);

CREATE TABLE IF NOT EXISTS transactions (
   txnid        INTEGER PRIMARY KEY AUTOINCREMENT,
   accno        INTEGER NOT NULL REFERENCES account(accno),
   txntype      TEXT    NOT NULL CHECK (txntype IN ('OPEN', 'DEPOSIT',
'WITHDRAW')),
   amount       REAL    NOT NULL CHECK (amount >= 0),
   balance_after REAL    NOT NULL,
   txntime      TEXT    NOT NULL DEFAULT (datetime('now', 'localtime'))
);
```

**Program (bank_app.py):**

```
import os
import sqlite3
import tkinter as tk
from tkinter import ttk, messagebox, simpledialog

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_FILE = os.path.join(BASE_DIR, "bank.db")
SCHEMA_FILE = os.path.join(BASE_DIR, "schema.sql")


class BankError(Exception):
   """Raised for any user-facing problem (bad input, no funds, ...)."""


class BankDB:
   def __init__(self, path=DB_FILE, schema_file=SCHEMA_FILE):
      self.conn = sqlite3.connect(path)
      self.conn.execute("PRAGMA foreign_keys = ON")   # enforce foreign keys
      try:
         with open(schema_file, encoding="utf-8") as f:
            self.conn.executescript(f.read())        # run schema.sql
      except FileNotFoundError:
         raise SystemExit(
            f"schema.sql not found ({schema_file}).\n"
         )
```

```
        cols = [r[1] for r in self.conn.execute("PRAGMA table_info(account)")]
        if "custid" not in cols:
            raise SystemExit(
                "An old bank.db from the single-table version was found.\n"
                "Delete bank.db and run the program again."
            )


    def all_accounts(self):
        return self.conn.execute(
            "SELECT a.accno, c.custid, c.cname, c.phone, a.balance "
            "FROM account a JOIN customer c ON a.custid = c.custid "
            "ORDER BY a.accno"
        ).fetchall()

    def get(self, accno):
        row = self.conn.execute(
            "SELECT a.accno, c.custid, c.cname, c.phone, a.balance "
            "FROM account a JOIN customer c ON a.custid = c.custid "
            "WHERE a.accno = ?", (accno,)
        ).fetchone()
        if row is None:
            raise BankError(f"Account {accno} does not exist.")
        return row

    def add(self, accno, cname, phone, balance):
        cname = cname.strip()
        balance = round(balance, 2)
        if not cname:
            raise BankError("Customer name cannot be empty.")
        if balance < 0:
            raise BankError("Opening balance cannot be negative.")
        exists = self.conn.execute(
            "SELECT 1 FROM account WHERE accno = ?", (accno,)
        ).fetchone()
        if exists:
            raise BankError(f"Account {accno} already exists.")
        try:
            with self.conn:
                cur = self.conn.execute(
                    "INSERT INTO customer (cname, phone) VALUES (?, ?)",
```

```
                (cname, phone or None),
            )
            self.conn.execute(
                "INSERT INTO account (accno, custid, balance) VALUES (?, ?, ?)",
                (accno, cur.lastrowid, balance),
            )
            self.conn.execute(
                "INSERT INTO transactions (accno, txntype, amount, balance_after) "
                "VALUES (?, 'OPEN', ?, ?)",
                (accno, balance, balance),
            )
    except sqlite3.IntegrityError as e:
        raise BankError(f"Could not save the record: {e}")

def update(self, accno, cname, phone):
    row = self.get(accno)
    cname = cname.strip()
    if not cname:
        raise BankError("Customer name cannot be empty.")
    with self.conn:
        self.conn.execute(
            "UPDATE customer SET cname = ?, phone = ? WHERE custid = ?",
            (cname, phone or None, row[1]),
        )

def delete(self, accno):
    row = self.get(accno)
    with self.conn:
        self.conn.execute("DELETE FROM transactions WHERE accno = ?", (accno,))
        self.conn.execute("DELETE FROM account WHERE accno = ?", (accno,))

        self.conn.execute(
            "DELETE FROM customer WHERE custid = ? "
            "AND NOT EXISTS (SELECT 1 FROM account WHERE custid = ?)",
            (row[1], row[1]),
        )

def deposit(self, accno, amount):
    return self._move(accno, amount, "DEPOSIT")

def withdraw(self, accno, amount):
```

```
        return self._move(accno, amount, "WITHDRAW")

    def _move(self, accno, amount, kind):
        amount = round(amount, 2)
        if amount <= 0:
            raise BankError(f"{kind.title()} amount must be greater than zero.")
        self.get(accno)
        change = amount if kind == "DEPOSIT" else -amount
        with self.conn:
            cur = self.conn.execute(
                "UPDATE account SET balance = ROUND(balance + ?, 2) "
                "WHERE accno = ? AND balance + ? >= 0",
                (change, accno, change),
            )
            if cur.rowcount == 0:
                raise BankError("Insufficient balance.")
            new_balance = self.conn.execute(
                "SELECT balance FROM account WHERE accno = ?", (accno,)
            ).fetchone()[0]
            self.conn.execute(
                "INSERT INTO transactions (accno, txntype, amount, balance_after) "
                "VALUES (?, ?, ?, ?)",
                (accno, kind, amount, new_balance),
            )
        return new_balance

    def history(self, accno=None):
        sql = ("SELECT txnid, accno, txntype, amount, balance_after, txntime "
               "FROM transactions ")
        if accno is None:
            return self.conn.execute(sql + "ORDER BY txnid DESC").fetchall()
        return self.conn.execute(
            sql + "WHERE accno = ? ORDER BY txnid DESC", (accno,)).fetchall()
def parse_accno(text):
    try:
        return int(text.strip())
    except ValueError:
        raise BankError("Account number must be a whole number.")


def parse_money(text, blank_ok=False):
```

```
    text = text.strip()
    if not text and blank_ok:
        return 0.0
    try:
        return float(text)
    except ValueError:
        raise BankError("Balance must be a number.")


def parse_phone(text):
    text = text.strip()
    if text and not (text.isdigit() and 7 <= len(text) <= 15):
        raise BankError("Phone number must be 7 to 15 digits.")
    return text

class BankApp(tk.Tk):
    def __init__(self, db):
        super().__init__()
        self.db = db
        self.title("Banking System")
        self.resizable(False, False)
        self.accno = tk.StringVar()
        self.cname = tk.StringVar()
        self.phone = tk.StringVar()
        self.balance = tk.StringVar()
        self._build()
        self.refresh()

    def _build(self):
        form = ttk.LabelFrame(self, text="Account details", padding=10)
        form.grid(row=0, column=0, padx=10, pady=(10, 4), sticky="ew")
        fields = [
            ("Account No", self.accno),
            ("Customer Name", self.cname),
            ("Phone", self.phone),
            ("Balance (Rs)", self.balance),
        ]
        for r, (label, var) in enumerate(fields):
            ttk.Label(form, text=label).grid(row=r, column=0, sticky="w", pady=3)
            ttk.Entry(form, textvariable=var, width=30).grid(row=r, column=1, padx=8)
```

```
        buttons = ttk.Frame(self)
        buttons.grid(row=1, column=0, padx=10, pady=4)
        actions = [
            ("Insert", self.insert),
            ("Update", self.update),
            ("Delete", self.delete),
            ("Clear", self.clear),
            ("Deposit", self.deposit),
            ("Withdraw", self.withdraw),
            ("History", self.show_history),
            ("Exit", self.destroy),
        ]
        for i, (text, cmd) in enumerate(actions):
            ttk.Button(buttons, text=text, command=cmd, width=10).grid(
                row=i // 4, column=i % 4, padx=3, pady=3
            )

        cols = ("accno", "custid", "cname", "phone", "balance")
        self.tree = ttk.Treeview(self, columns=cols, show="headings", height=8)
        for col, title, width in [
            ("accno", "Account No", 90),
            ("custid", "Cust ID", 60),
            ("cname", "Customer Name", 160),
            ("phone", "Phone", 110),
            ("balance", "Balance (Rs)", 100),
        ]:
            self.tree.heading(col, text=title)
            self.tree.column(col, width=width, anchor="w")
        self.tree.grid(row=2, column=0, padx=10, pady=(4, 10))
        self.tree.bind("<<TreeviewSelect>>", self.on_select)
    def refresh(self):
        self.tree.delete(*self.tree.get_children())
        for accno, custid, cname, phone, balance in self.db.all_accounts():
            self.tree.insert("", "end",
                             values=(accno, custid, cname, phone or "", f"{balance:.2f}"))

    def on_select(self, _event=None):
        sel = self.tree.selection()
        if sel:
            accno, _custid, cname, phone, balance = self.tree.item(sel[0], "values")
            self.accno.set(accno)
```

```
        self.cname.set(cname)
        self.phone.set(phone)
        self.balance.set(balance)

    def error(self, err):
        messagebox.showerror("Banking System", str(err), parent=self)

    def info(self, msg):
        messagebox.showinfo("Banking System", msg, parent=self)

    def ask_amount(self, title, prompt):
        return simpledialog.askfloat(title, prompt, parent=self, minvalue=0.01)

    def insert(self):
        try:
            accno = parse_accno(self.accno.get())
            balance = parse_money(self.balance.get(), blank_ok=True)
            phone = parse_phone(self.phone.get())
            self.db.add(accno, self.cname.get(), phone, balance)
        except BankError as e:
            return self.error(e)
        self.refresh()
        self.info("Record inserted successfully.")

    def update(self):
        try:
            accno = parse_accno(self.accno.get())
            phone = parse_phone(self.phone.get())
            self.db.update(accno, self.cname.get(), phone)
        except BankError as e:
            return self.error(e)
        self.refresh()
        self.info("Record updated successfully. "
                  "(Balance changes only through Deposit / Withdraw.)")

    def delete(self):
        try:
            accno = parse_accno(self.accno.get())
            self.db.get(accno)
            if not messagebox.askyesno(
                    "Banking System",
```

```
                f"Delete account {accno} and its transaction history?",
                parent=self):
            return
        self.db.delete(accno)
    except BankError as e:
        return self.error(e)
    self.clear()
    self.refresh()
    self.info("Record deleted.")

def clear(self):
    for var in (self.accno, self.cname, self.phone, self.balance):
        var.set("")
    self.tree.selection_remove(self.tree.selection())

def deposit(self):
    self._transact("Deposit", "Enter the amount to be deposited (Rs):",
                   self.db.deposit)

def withdraw(self):
    self._transact("Withdraw", "Enter the amount to be withdrawn (Rs):",
                   self.db.withdraw)

def _transact(self, title, prompt, operation):
    try:
        accno = parse_accno(self.accno.get())
        self.db.get(accno)
        amount = self.ask_amount(title, prompt)
        if amount is None:
            return
        new_balance = operation(accno, amount)
    except BankError as e:
        return self.error(e)
    self.refresh()
    self.balance.set(f"{new_balance:.2f}")
    self.info(f"Current balance is Rs {new_balance:.2f}")

def show_history(self):
    """Transaction history of the account in the box (all accounts if blank)."""
    try:
        text = self.accno.get().strip()
```

```
            accno = parse_accno(text) if text else None
            if accno is not None:
                self.db.get(accno)
            rows = self.db.history(accno)
        except BankError as e:
            return self.error(e)

        win = tk.Toplevel(self)
        win.title(f"Transaction history - account {accno}" if accno is not None
                  else "Transaction history - all accounts")
        win.resizable(False, False)
        cols = ("txnid", "accno", "txntype", "amount", "balance_after", "txntime")
        tree = ttk.Treeview(win, columns=cols, show="headings", height=12)
        for col, title, width in [
            ("txnid", "Txn ID", 60),
            ("accno", "Account No", 90),
            ("txntype", "Type", 80),
            ("amount", "Amount (Rs)", 100),
            ("balance_after", "Balance After", 100),
            ("txntime", "Date & Time", 150),
        ]:
            tree.heading(col, text=title)
            tree.column(col, width=width, anchor="w")
        for txnid, acc, kind, amount, after, when in rows:
            tree.insert("", "end",
                        values=(txnid, acc, kind, f"{amount:.2f}", f"{after:.2f}", when))
        tree.grid(row=0, column=0, padx=10, pady=10)
        ttk.Button(win, text="Close", command=win.destroy).grid(row=1, column=0,
pady=(0, 10))


if __name__ == "__main__":
    BankApp(BankDB()).mainloop()
```

**Output:**

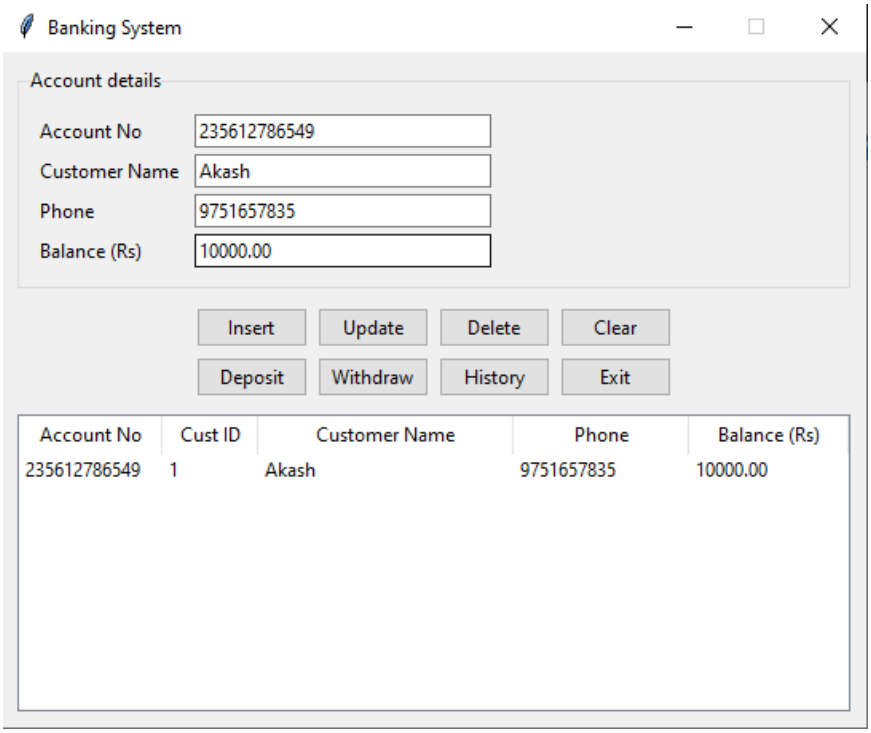



*Fig. 12.3: Main window after Insert, showing the new account with its opening balance of Rs 10,000.00.*

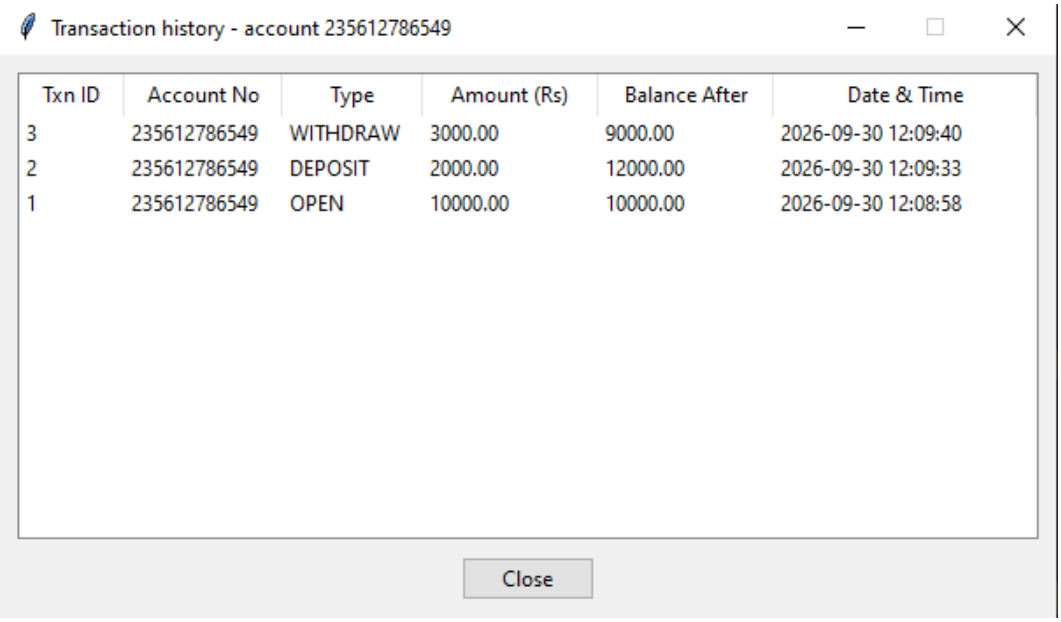



*Fig. 12.4: Transaction history of the same account: OPEN Rs 10,000.00, DEPOSIT Rs 2,000.00, WITHDRAW Rs 3,000.00, final balance Rs 9,000.00.*

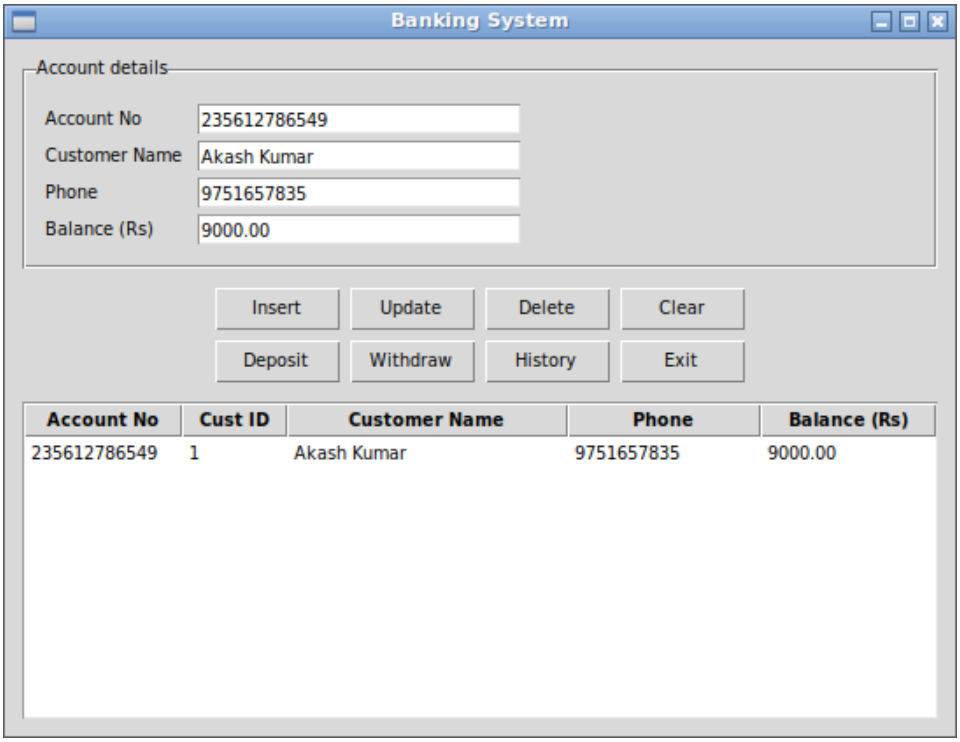



*Fig. 12.5: Main window after Update: the customer name is changed to "Akash Kumar"; the balance stays Rs 9,000.00 because only Deposit and Withdraw change it.*

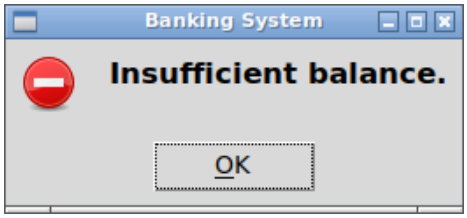



*Fig. 12.6: Withdraw of Rs 20,000.00 is rejected with the "Insufficient balance." message; the balance is not changed.*

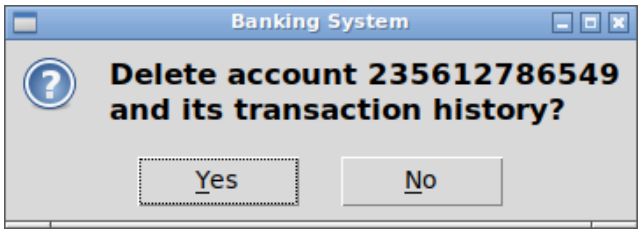



*Fig. 12.7: Confirmation prompt shown when Delete is clicked.*

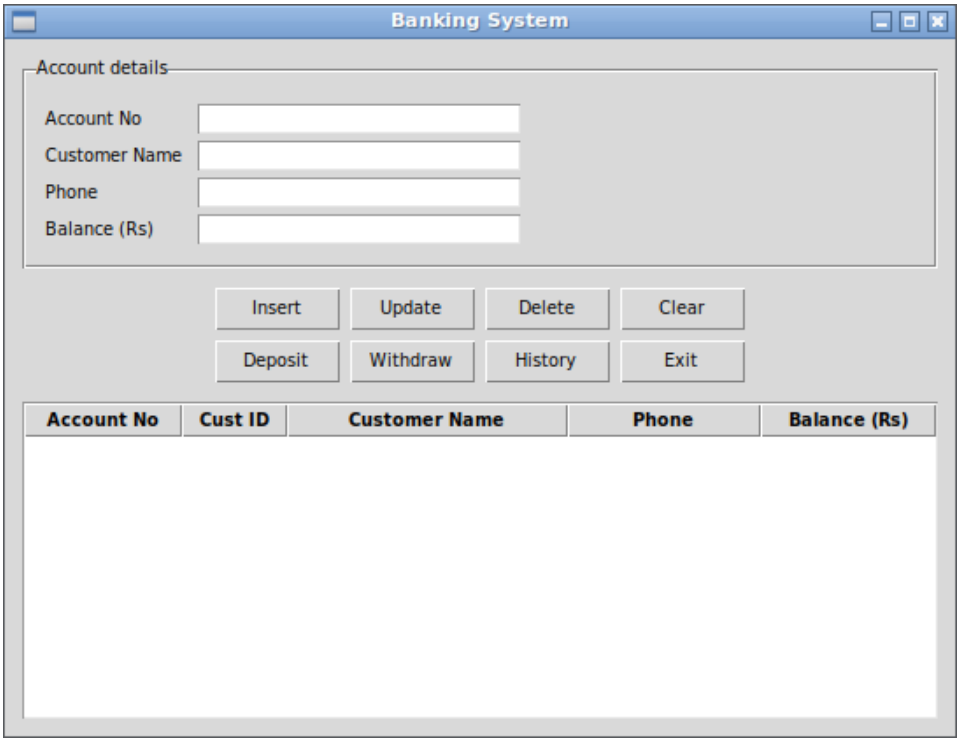



*Fig. 12.8: Main window after the account is deleted; the account list is empty.*

## Ex. No: 13

```
BEGIN
  DBMS_XMLSCHEMA.registerSchema(
    schemaurl => 'my_schema.xsd',
    schemadoc => '<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
elementFormDefault="qualified">
<xs:element name="shiporder"><xs:complexType><xs:sequence>
<xs:element name="orderperson" type="xs:string"/>
<xs:element name="shipto"><xs:complexType><xs:sequence>
<xs:element name="name" type="xs:string"/>
<xs:element name="address" type="xs:string"/>
<xs:element name="city" type="xs:string"/>
<xs:element name="country" type="xs:string"/>
</xs:sequence></xs:complexType></xs:element>
<xs:element name="item"
maxOccurs="unbounded"><xs:complexType><xs:sequence>
<xs:element name="title" type="xs:string"/>
<xs:element name="note" type="xs:string" minOccurs="0"/>
<xs:element name="quantity" type="xs:positiveInteger"/>
<xs:element name="price" type="xs:decimal"/>
</xs:sequence></xs:complexType></xs:element>
</xs:sequence><xs:attribute name="orderid" type="xs:string"
use="required"/></xs:complexType></xs:element>
</xs:schema>',
    local     => TRUE,
    gentypes  => FALSE,
    gentables => FALSE);
END;
/
```

**Output:**

*PL/SQL procedure successfully completed.*

```
CREATE TABLE t1 (id NUMBER, xml_doc XMLTYPE);
```

**Output:**

*Table created.*

```
INSERT INTO t1 VALUES (1, XMLTYPE(q'[<?xml version="1.0" encoding="UTF-
8"?>
<shiporder orderid="889923">
<orderperson>John Smith</orderperson>
```

<shipto><name>Ola Nordmann</name><address>Langgt 23</address><city>4000 Stavanger</city><country>Norway</country></shipto>

<item><title>Empire Burlesque</title><note>Special Edition</note><quantity>1</quantity><price>10.90</price></item>

<item><title>Hide your heart</title><quantity>1</quantity><price>9.90</price></item>

</shiporder>]'));

INSERT INTO t1 VALUES (2, XMLTYPE(q'[<?xml version="1.0" encoding="UTF-8"?>

<shiporder orderid="889923">

<orderperson>John Smith</orderperson>

<shipto><name1>Ola Nordmann</name1><address>Langgt 23</address><city>4000 Stavanger</city><country>Norway</country></shipto>

<item><title>Empire Burlesque</title><note>Special Edition</note><quantity>1</quantity><price>10.90</price></item>

<item><title>Hide your heart</title><quantity>1</quantity><price>9.90</price></item>

</shiporder>]'));

**Output:**

*2 rows created.*

COMMIT;

**Output:**

*Commit complete.*

SELECT t.id, t.xml_doc.isSchemaValid('my_schema.xsd') AS is_valid FROM t1 t;

**Output:**

| ID | IS_VALID |
|---|---|
| 1 | 1 |
| 2 | 0 |

## Ex. No: 14

CREATE TABLE employeepersonaldetails (id INT PRIMARY KEY, name VARCHAR(30) NOT NULL,
age INT NOT NULL, mobilenumber NUMBER(10) NOT NULL UNIQUE);

**Output:**

*Table created.*

INSERT INTO employeepersonaldetails VALUES (1, 'Ajay',  20, 9876543210);
INSERT INTO employeepersonaldetails VALUES (2, 'Brindha', 20, 9874563210);
INSERT INTO employeepersonaldetails VALUES (3, 'Kumaran', 20, 9673443210);

**Output:**

*3 rows created.*

SELECT * FROM employeepersonaldetails;

**Output:**

| ID | NAME | AGE | MOBILE NUMBER |
|---|---|---|---|
| 1 | Ajay | 20 | 9876543210 |
| 2 | Brindha | 20 | 9874563210 |
| 3 | Kumaran | 20 | 9673443210 |

COMMIT;

**Output:**

*Commit complete.*

INSERT INTO employeepersonaldetails VALUES (4, 'Archana', 20, 9876543410);

**Output:**

*1 row created.*

SELECT * FROM employeepersonaldetails;

**Output:**

| ID | NAME | AGE | MOBILE NUMBER |
|---|---|---|---|
| 1 | Ajay | 20 | 9876543210 |
| 2 | Brindha | 20 | 9874563210 |
| 3 | Kumaran | 20 | 9673443210 |
| 4 | Archana | 20 | 9876543410 |

ROLLBACK;

**Output:**

*Rollback complete.*

SELECT * FROM employeepersonaldetails;

**Output:**

| ID | NAME | AGE | MOBILE NUMBER |
|---|---|---|---|
| 1 | Ajay | 20 | 9876543210 |
| 2 | Brindha | 20 | 9874563210 |
| 3 | Kumaran | 20 | 9673443210 |

INSERT INTO employeepersonaldetails VALUES (5, 'Braham', 20, 9878953210);

**Output:**

*1 row created.*

SAVEPOINT a;

**Output:**

*Savepoint created.*

INSERT INTO employeepersonaldetails VALUES (6, 'Srimathi', 20, 9678953210);

**Output:**

*1 row created.*

SAVEPOINT b;

**Output:**

*Savepoint created.*

SELECT * FROM employeepersonaldetails;

**Output:**

| ID | NAME | AGE | MOBILE NUMBER |
|---|---|---|---|
| 1 | Ajay | 20 | 9876543210 |
| 2 | Brindha | 20 | 9874563210 |
| 3 | Kumaran | 20 | 9673443210 |
| 5 | Braham | 20 | 9878953210 |
| 6 | Srimathi | 20 | 9678953210 |

ROLLBACK TO a;

**Output:**

*Rollback complete.*

SELECT * FROM employeepersonaldetails;

**Output:**

| ID | NAME | AGE | MOBILE NUMBER |
|---|---|---|---|
| 1 | Ajay | 20 | 9876543210 |
| 2 | Brindha | 20 | 9874563210 |
| 3 | Kumaran | 20 | 9673443210 |
| 5 | Braham | 20 | 9878953210 |

COMMIT;

**Output:**

*Commit complete.*

CREATE TABLE account (accno INT PRIMARY KEY, cname VARCHAR(20), balance NUMBER(10,2));

**Output:**

*Table created.*

INSERT INTO account VALUES (101, 'Ajay', 5000);
INSERT INTO account VALUES (102, 'Brindha', 3000);

**Output:**

*2 rows created.*

COMMIT;

**Output:**

*Commit complete.*

UPDATE account SET balance = balance - 1000 WHERE accno = 101;

**Output:**

*1 row updated.*

SAVEPOINT debited;

**Output:**

*Savepoint created.*

UPDATE account SET balance = balance + 1000 WHERE accno = 102;

**Output:**

*1 row updated.*

SELECT * FROM account;

**Output:**

| ACCNO | CNAME | BALANCE |
|---|---|---|
| 101 | Ajay | 4000 |
| 102 | Brindha | 4000 |

ROLLBACK TO debited;

**Output:**

*Rollback complete.*

SELECT * FROM account;

**Output:**

| ACCNO | CNAME | BALANCE |
|---|---|---|
| 101 | Ajay | 4000 |
| 102 | Brindha | 3000 |

ROLLBACK;

**Output:**

*Rollback complete.*

SELECT * FROM account;

**Output:**

| ACCNO | CNAME | BALANCE |
|---|---|---|
| 101 | Ajay | 5000 |
| 102 | Brindha | 3000 |

CREATE USER student2 IDENTIFIED BY student2pwd;

**Output:**

*User created.*

GRANT CREATE SESSION TO student2;

**Output:**

*Grant succeeded.*

CONNECT student2/student2pwd

**Output:**

*Connected.*

SELECT * FROM system.employeepersonaldetails;

**Output:**

*ERROR at line 1:*
*ORA-00942: table or view does not exist*

CONNECT system/&&sys_pwd

**Output:**

*Connected.*

GRANT SELECT ON employeepersonaldetails TO student2;

**Output:**

*Grant succeeded.*

SELECT grantee, table_name, privilege FROM user_tab_privs_made WHERE grantee = 'STUDENT2';

**Output:**

| GRANTEE | TABLE_NAME | PRIVILEGE |
|---|---|---|
| STUDENT2 | EMPLOYEEPERSONALDETAILS | SELECT |

CONNECT student2/student2pwd

**Output:**

*Connected.*

SELECT * FROM system.employeepersonaldetails;

**Output:**

| ID | NAME | AGE | MOBILE NUMBER |
|---|---|---|---|
| 1 | Ajay | 20 | 9876543210 |
| 2 | Brindha | 20 | 9874563210 |
| 3 | Kumaran | 20 | 9673443210 |
| 5 | Braham | 20 | 9878953210 |

INSERT INTO system.employeepersonaldetails VALUES (7, 'Test', 20, 9000000001);

**Output:**

*ERROR at line 1:*
*ORA-01031: insufficient privileges*

CONNECT system/&&sys_pwd

**Output:**

*Connected.*

GRANT INSERT ON employeepersonaldetails TO student2;

**Output:**

*Grant succeeded.*

CONNECT student2/student2pwd

**Output:**

*Connected.*

INSERT INTO system.employeepersonaldetails VALUES (7, 'Test', 20, 9000000001);

**Output:**

*1 row created.*

ROLLBACK;

**Output:**

*Rollback complete.*

CONNECT system/&&sys_pwd

**Output:**

*Connected.*

REVOKE INSERT ON employeepersonaldetails FROM student2;

**Output:**

*Revoke succeeded.*

REVOKE SELECT ON employeepersonaldetails FROM student2;

**Output:**

*Revoke succeeded.*

CONNECT student2/student2pwd

**Output:**

*Connected.*

SELECT * FROM system.employeepersonaldetails;

**Output:**

*ERROR at line 1:*
*ORA-00942: table or view does not exist*